

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgavi – 590018.



Report on

“Design a DFA for given language using JFLAP software”

Submitted in partial fulfillment of the requirements of the award of degree of

**BACHELOR OF ENGINEERING
IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

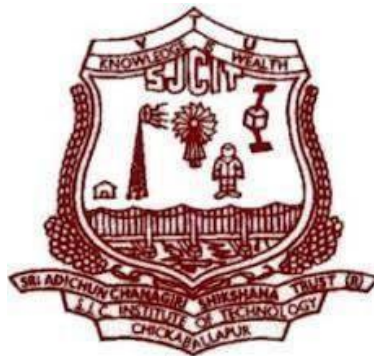
Submitted By:

MAREDDY CHARAN

1SJ22AD024

UNDER THE GUIDANCE OF

**SANDHYA R
ASSISTANT PROFESSOR
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE, SJCIT**



SJC INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE, CHICKBALLAPUR-562101

2024-2025

Introduction

JFLAP (Java Formal Languages and Automata Package) is an interactive software tool designed to help students and researchers explore concepts in formal languages, automata theory, and computational theory. It provides a graphical interface for experimenting with various computational models such as:

- **Finite Automata** (Deterministic and Nondeterministic)
- **Context-Free Grammars (CFGs)**
- **Pushdown Automata (PDA)**
- **Turing Machines**
- **Regular Expressions**

Key Features:

- **Visualization:** JFLAP allows users to create and visualize automata, grammars, and machines, which aids in understanding the behaviour of these formal systems.
- **Simulations:** It can simulate the operation of automata on input strings, making it easier to understand how a machine processes data.
- **Conversion Tools:** JFLAP includes tools for converting between different representations (e.g., from regular expressions to finite automata, or vice versa).
- **Learning Aid:** It is widely used in educational settings to teach theory of computation and formal languages.

JFLAP is a helpful tool for anyone studying automata theory, as it provides an interactive, visual environment to experiment and test concepts that can be abstract and difficult to grasp through theory alone.

Official Website of JFlap - <https://www.jflap.org/>

Steps to install JFLAP

1. Visit the website <https://www.jflap.org/>



2. Press Get JFLAP

Get JFLAP

INFORMATION about JFLAP:

- Get JFLAP Software

Please [fill out this form](#) and you can have the most recent version of JFLAP to use for free.

3. Fill the form

Get JFLAP

INFORMATION about JFLAP:

- Get JFLAP Software

Please [fill out this form](#) and you can have the most recent version of JFLAP to use for free.

4. After filling the form press Submit.

JFLAP Information

Thank you for providing your information to the Department of Computer Science at Duke University. We appreciate your interest in our programs.

Your data submission for the form has been successfully processed!

You will now be redirected in a few seconds. (click [here](#) to be redirected immediately)

Department of Computer Science
Box 90129
Duke University
Durham, NC 27708-0129
(919) 660-6500

5. Press the following link to download the '.jar' file.

To get JFLAP SOFTWARE (jar file):

- JFLAP Software - will put newest version here as updates occur

This is a .jar file.

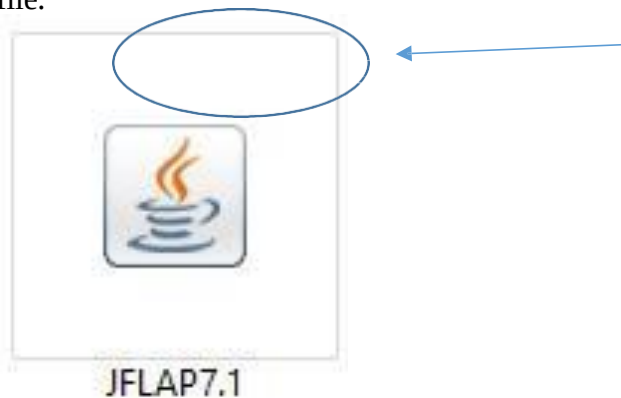
- Jul 27, 2018 - JFLAP Version 7.1
 - Note: JFLAP now uses Java 8.0.
 - [Changes made to JFLAP 7.0](#)
 - [JFLAP7.1.jar](#)

6. Pre-requisite of running JFLAP is installations of JDK.

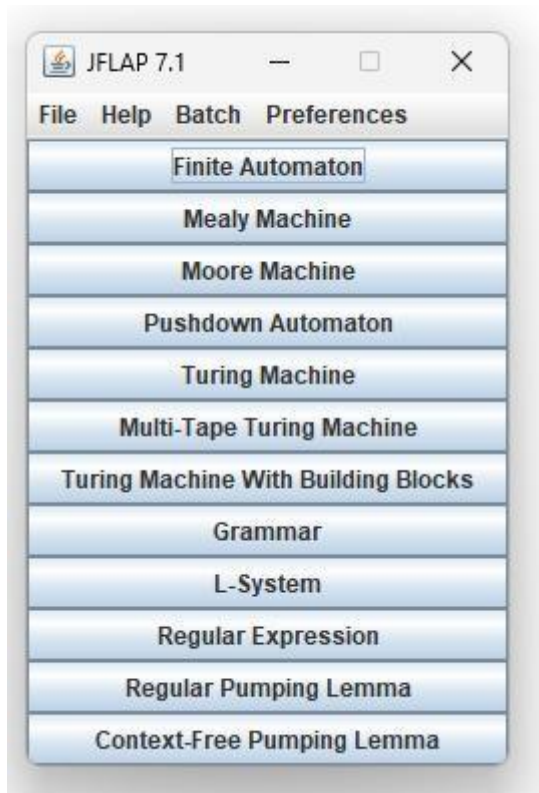


7. After successfully
file.

downloading the file. Locate the

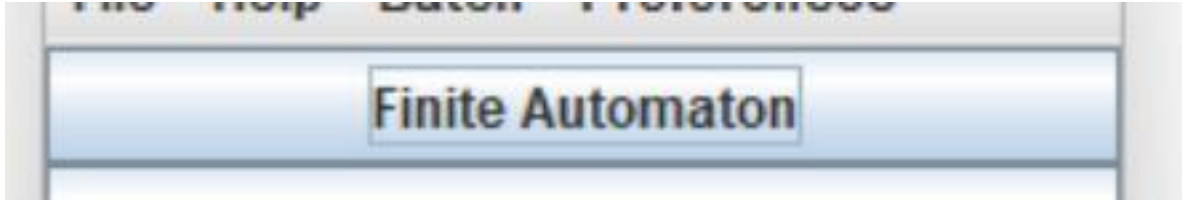


Now open the file by double-clicking the file.

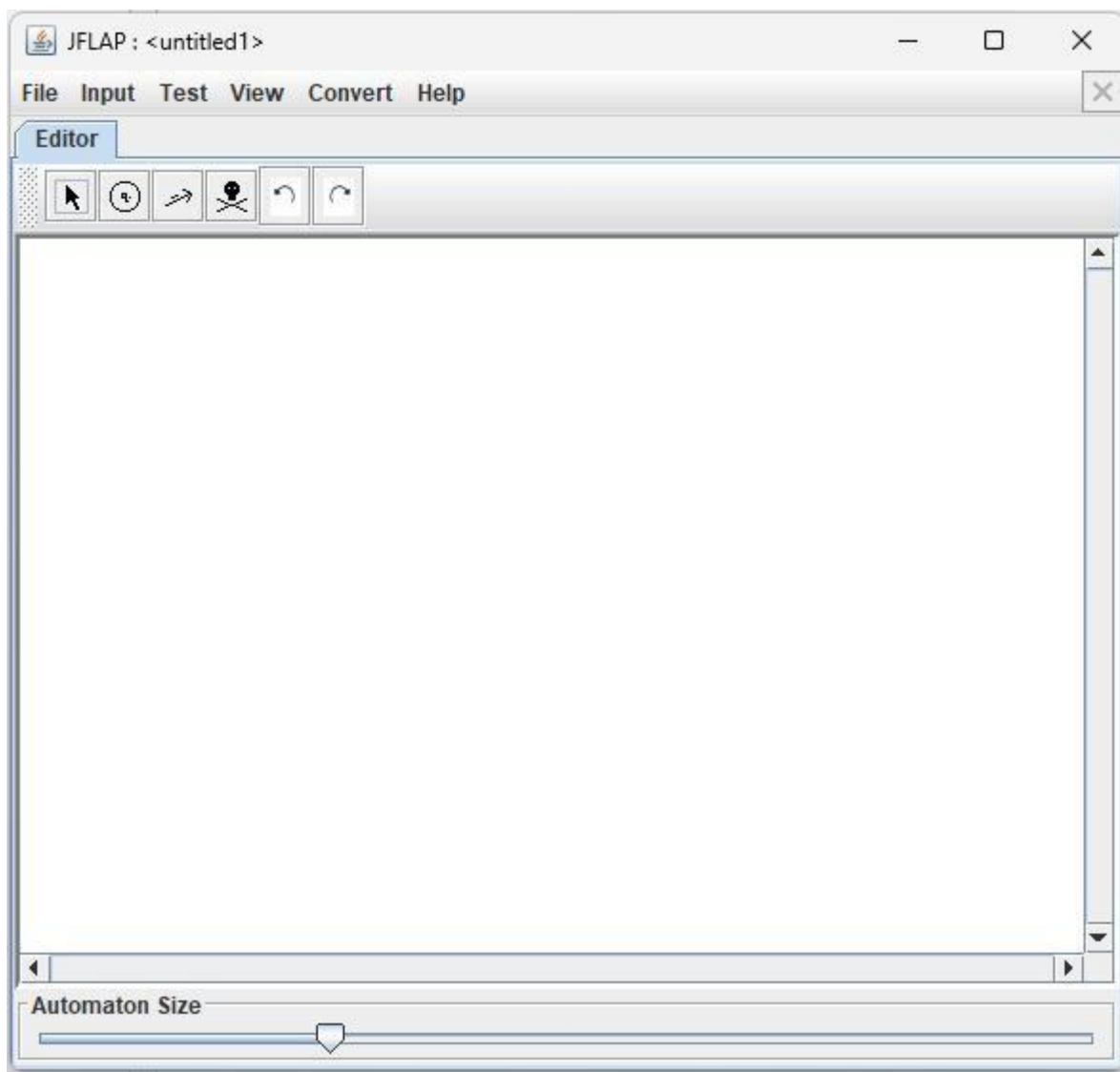


This popup will appear, now select the appropriate application and proceed.

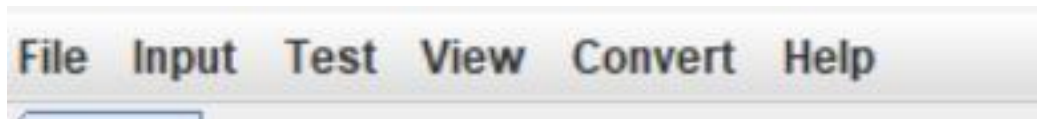
Understanding Automata Editor



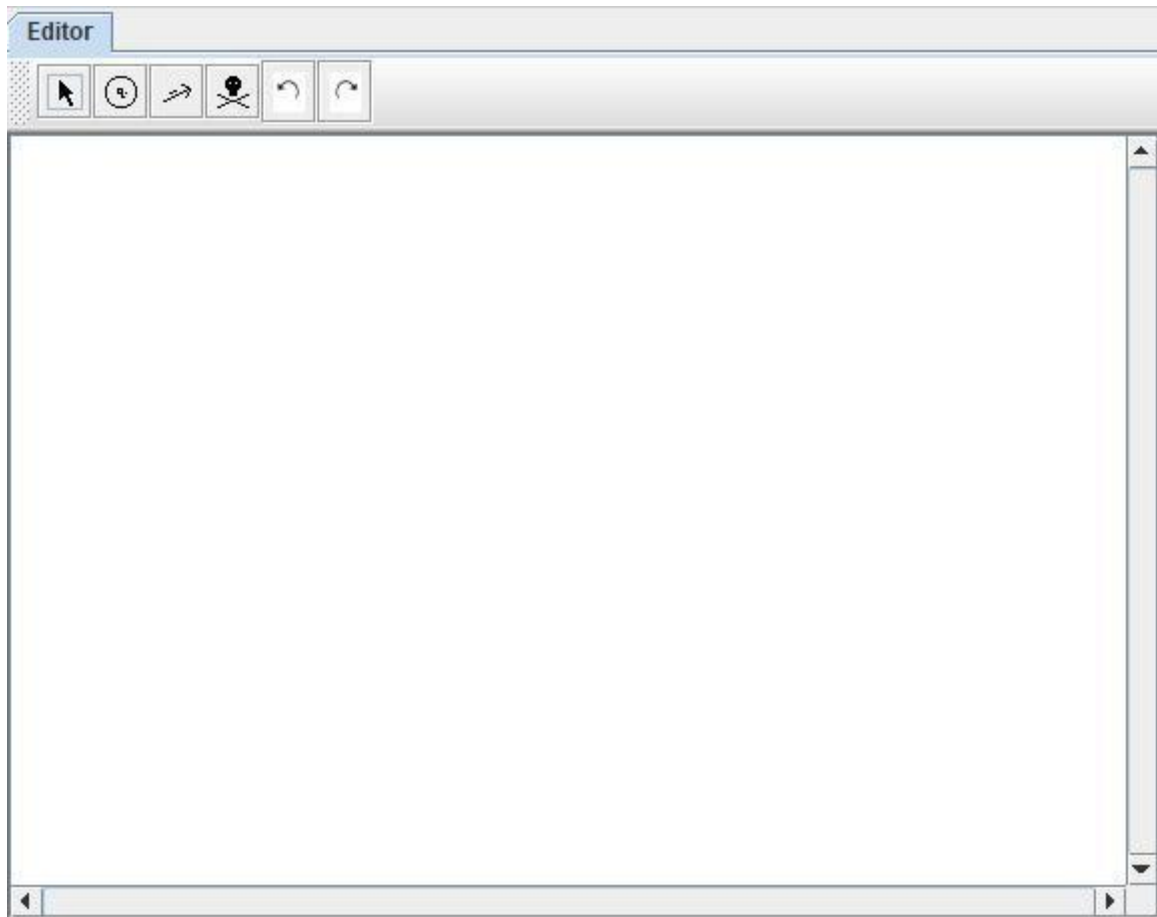
After selecting this Finite automata, the following window will open, this window contains multiple options, which directly or indirectly are very useful.



The menu contains 6 options, which further contain sub-menu.



The Input option contains some very important set of functions, which will be used in the future implementation.



The above is the editing module, where the actual FA (Finite Automata) can be designed.

The editor module has 6 options,



Option 1- Arrow key:

This function is used for adjusting and relocating the already existing states i.e. drag and drop.

Option 2 - State selector:

This function is used for creating states, using the left click on the mouse the states can be put on the editor section.

Option 3 – Transition Creator:

This option can be utilized to connect two states by creating a state transition with any symbol.

Option 4 – Delete:

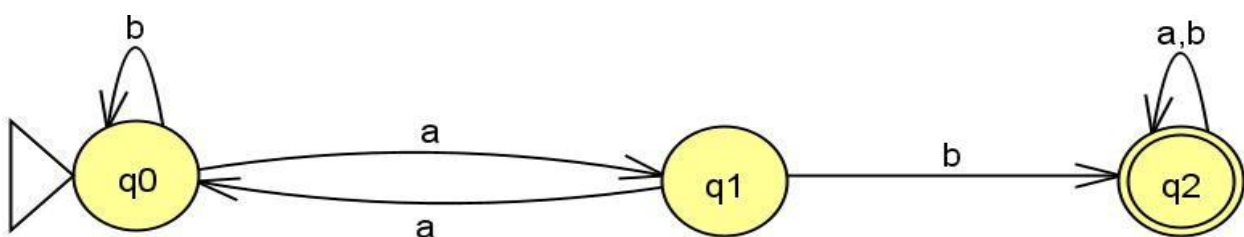
This function can be utilized to delete any state or transition.

Option 5&6 – Undo and Redo.

Simulating an Automata

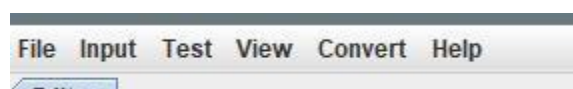
Example: Construct a DFA for a string of a's and b's containing the substring 'ab'.

The above is the constructed DFA for the given question.



Now, let's test the DFA with a string to check whether it satisfies the given condition of substring 'ab'.

To test the substring, go to the 'input' option in the drop-down menu tabs.

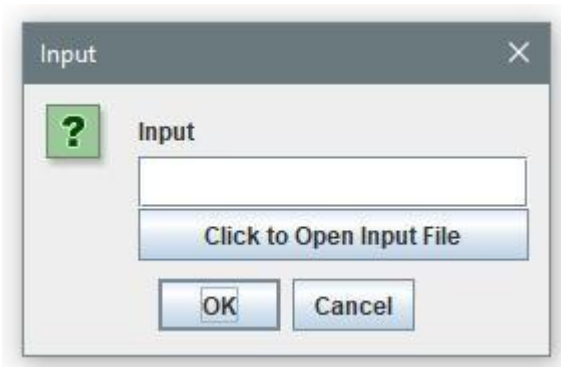


Then select 'Step by State' option,



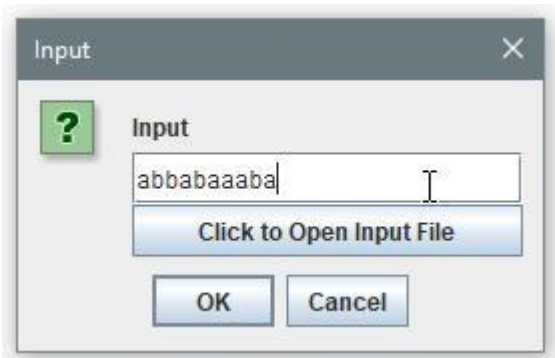
After selecting that option,

you'll get a pop to enter a string, just like the one in the bellow image.

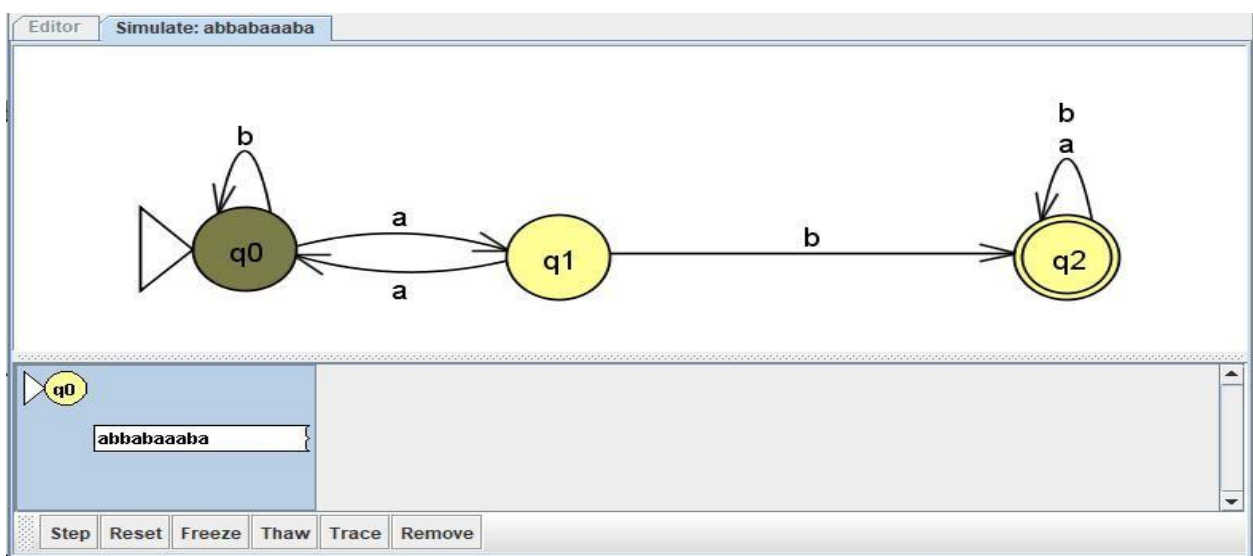


Now enter the string to be checked in the dialog box in the pop-up.

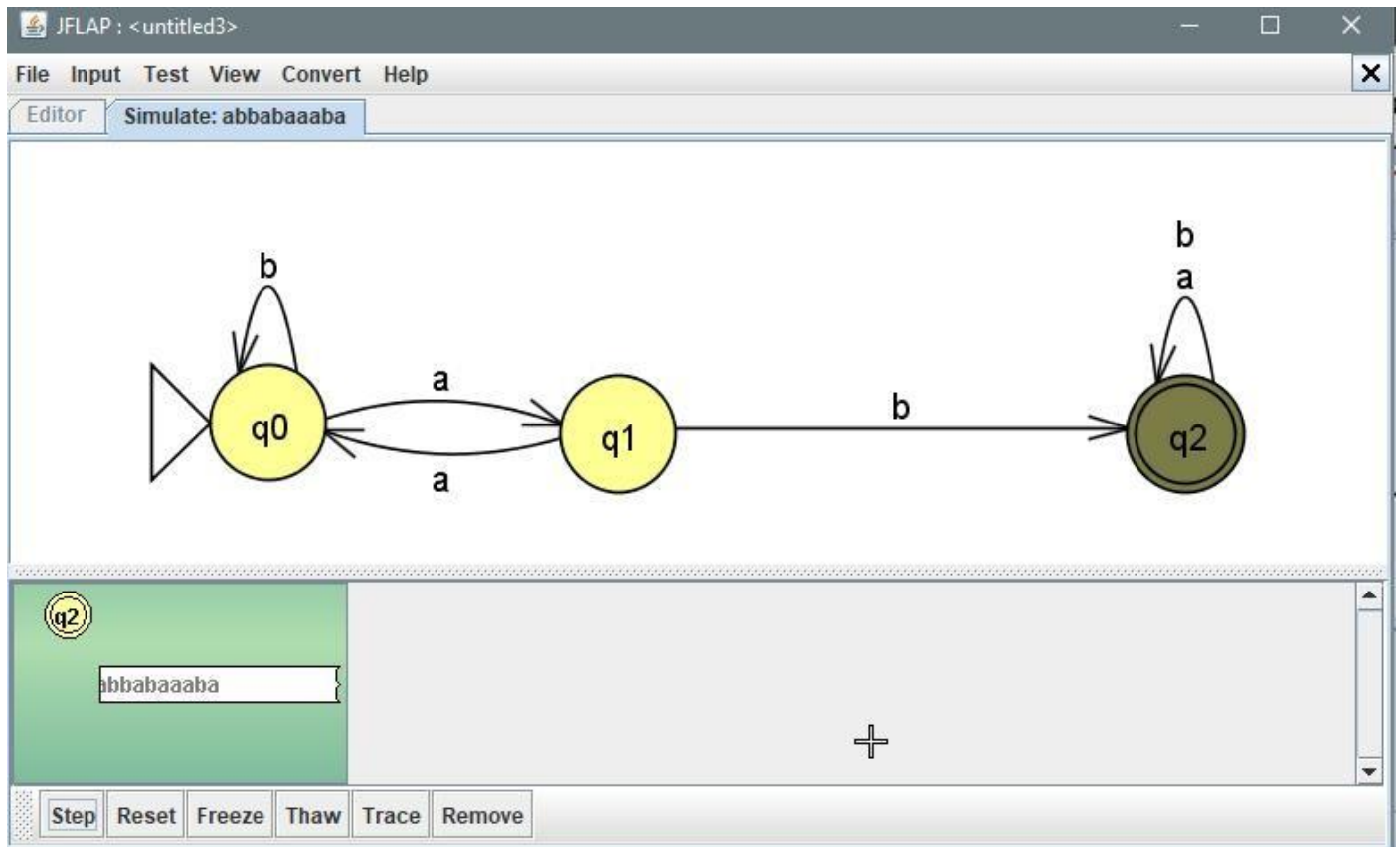
Then press 'Click to open input File', bellow is the input string to test the given DFA.



After this step, you'll be redirected back to the editor tab, and another new tab will open by the title 'Simulate:<your_string>'



In the left bottom most corner the option 'step' is available, using that option will be studying the moment of state transition of string on the constructed DFA.



After press 'step' at the last most symbol of the string, if the box turns 'green', it means the string is satisfied with the constructed DFA.

If the box turns 'red', like the image down, it means the string failed to match the DFA.



We have successfully simulated a DFA on JFLap, an opensource java-based tool.

“Conversion of Regular Expression to Finite Automata”

Create a New Regular Expression:

From the main menu, go to File > New and select "Regular Expression".

Enter the Regular Expression:

In the Regular Expression window, type or paste your regular expression in the text box. For example, for a regular expression like $a(b|c)^*d$, simply type it in.

Convert the Regular Expression to an NFA (Non-deterministic Finite Automaton):

After entering the regular expression, click on the Convert button (or go to the menu Convert > To NFA).

JFLAP will automatically convert your regular expression into a Non-deterministic Finite Automaton (NFA).

View the NFA:

The NFA will appear on the screen, with states and transitions representing the regular expression.

You can interact with the NFA by clicking and dragging states, or by selecting specific transitions.

Save and Export:

Save your work using the File > Save As option. Files are saved with .jff extension.

Export diagrams or results for reports or presentations as images or text.

Use Advanced Features:

For more complex tasks, such as:

Minimization of DFA

Grammar parsing and ambiguity checking Multi-tape Turing Machines

Feel free to ask if you need specific examples or guidance on using JFLAP.

To convert the regular expression $(a+b)^*abb$ into a finite automaton (FA), we'll break it down into smaller parts and then combine them. Here's how we can approach it:

Regular Expression: $(a+b)^*abb$

1. $(a+b)^*$: This part matches any sequence of as and bs, including the empty string.
2. abb : This part matches the exact sequence "abb" at the end.

Step-by-Step Construction of the Finite Automaton

We will construct a **Nondeterministic Finite Automaton (NFA)** that recognizes $(a+b)^*abb$, as an NFA is simpler to create for regular expressions with repetition (like $(a+b)^*$).

States:

- **q0**: The start state, which can loop on any sequence of as or bs, or move toward

recognizing the sequence abb.

- **q1**: This state is reached after reading the first a of abb.
- **q2**: This state is reached after reading the first b of abb.
- **q3**: This is the accepting state, which is reached after reading the second b of abb.

Alphabet:

The alphabet is $\Sigma=\{a,b\}$

Transitions:

1. From **q0** (the start state):
 - On input a, stay in **q0** (since $(a+b)^*$ can match any sequence of as and bs).
 - On input b, stay in **q0** (same reason as above).
 - On input a, transition to **q1** (start matching "abb").
2. From **q1** (after reading the first a of "abb"):
 - On input a, stay in **q1** (any sequence of as can precede "abb").
 - On input b, transition to **q2** (read the first b of "abb").
3. From **q2** (after reading the first b of "abb"):
 - On input a, return to **q0** (any a resets the matching process).
 - On input b, transition to **q3** (read the second b of "abb").
4. From **q3** (after reading the second b of "abb"):
 - On input a, stay in **q0** (we can continue matching additional as and bs).
 - On input b, stay in **q0** (same reason as above).

Accepting States:

- **q3** is the accepting state because this is the state reached when the automaton has successfully matched the sequence "abb" at the end of the string.

Final NFA Description:

- **States:** $Q=\{q_0,q_1,q_2,q_3\}$
- **Alphabet:** $\Sigma=\{a,b\}$
- **Start state:** q_0
- **Accept state:** q_3
- **Transitions:**

Current State Input a Input b

q0 q0 q0

q0 q1 -

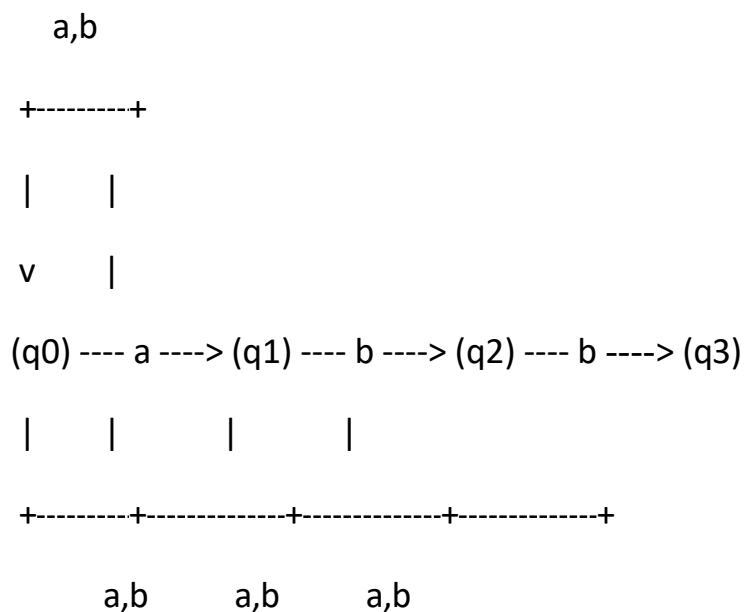
q1 q1 q2

q2 q0 q3

q3 q0 q0

Diagram of the NFA

Here's the diagram for the NFA:



Explanation:

- **q0** is the initial state. From **q0**, we can loop on any sequence of as and bs. This accounts for $(a+b)^*$, which allows any combination of as and bs.
- When we encounter the first a, we transition to **q1** (starting the sequence "abb").
- After reading the first b, we move to **q2**.
- After reading the second b, we reach **q3**, the accepting state, because we've matched the sequence "abb".
- From **q3**, we can continue reading any as or bs and the automaton will remain in **q0**.

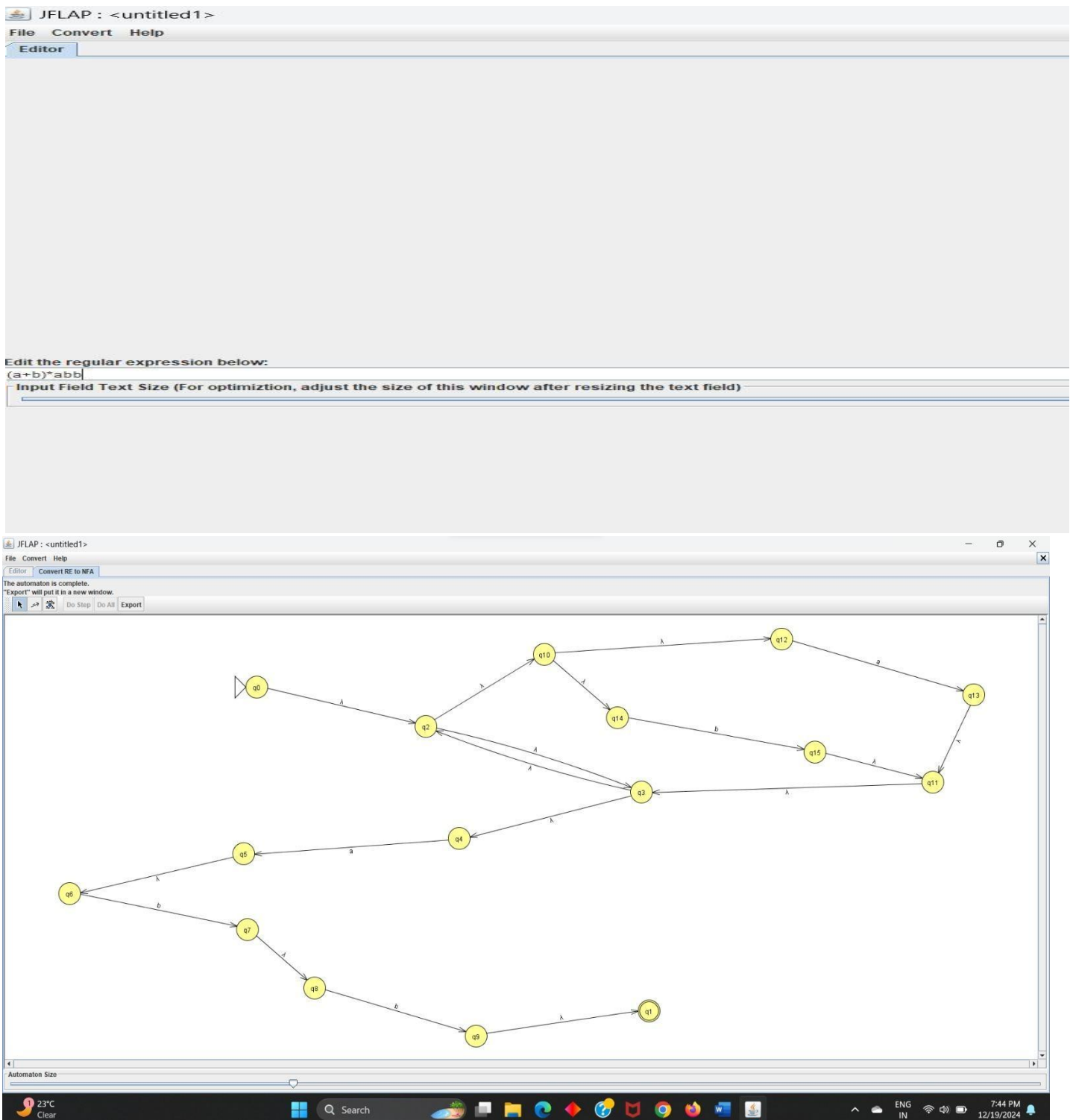
Final Notes:

- This is an **NFA** because it has non-deterministic transitions where the automaton can

stay in **q0** or transition to other states when reading a or b.

- The automaton accepts any string that ends with "abb" and allows arbitrary sequences of as and bs before it.

This NFA matches the regular expression $(a+b)^*abb$.



We have successfully simulated a Regular Expression on JFLap, an opensource java-based tool.